VERYL2CHIP · 範例分析

蜂鳴器音樂分析報告

《小星星》— 用 RISC-V SoC 的 LED 腳位驅動蜂鳴器

來源：veryl2chip IDE | 範例 ID：c_music | 目標板：iCEBreaker（iCE40UP5K） | 日期：2026-10-29

摘 摘要

本報告分析 veryl2chip IDE 範例庫中的「**C：蜂鳴器音樂**」（`c_music`）。範例以 C 語言在 RvSoc（RISC-V 多週期 SoC） 上執行，把 `0x1000_0000` 的 LED 暫存器 bit 0 當成方波輸出，經 PMOD1A 腳 1 推動蜂鳴器， 演奏《小星星》的前兩句。

除了原始碼分析，本報告也依相同頻率與時間參數**重建了實際音檔**，並以過零點量測驗證音高。 旋律共 16 個事件（14 個音 + 2 個休止），每個音 300 ms 加 50 ms 間隔，全長 **5.60 秒**； 實測頻率與程式計算值的誤差小於 0.03 Hz。

5.60 s	16	6	< 57 ppm
全曲長度	音符 / 休止事件	使用音高（C4–A4）	頻率量化誤差

音檔：48 kHz / 16-bit / 單聲道，268,800 個取樣，5.60 秒。以程式實際輸出的頻率重建，非另外配音。

1 取得方式與專案內容

在 IDE 中打開左側 📚 **範例庫**，分類選 **「C 程式」**，點一下 **「C：蜂鳴器音樂」**， 就會開成一個新專案（修改只存在瀏覽器，不會動到範例）。

- 範例說明：*PMOD1A 腳 1 的方波（蜂鳴器模組）：小星星*
- 合成頂層：`RvBoard`；模擬／執行模式：`cpu`（UART 115200）
- 板子設定：`"pmod1a": "buzzer"`、`"pmod1b": ""`
- PMOD1A 的 8 個輸出對應 `o_led[7:0]`；蜂鳴器接在**腳 1 =** `o_led[0]`（iCE40 腳位 4）

檔案	用途	大小
`music.c`	旋律與方波產生（本報告主角）	720 B
`rvsoc.veryl`	多週期 RISC-V SoC：CPU、RAM、LED／UART／計時器	36,980 B
`rv32i.veryl`	RV32I 核心與 ALU	13,115 B
`sync_ram.veryl`	同步讀取 RAM（合成後變 SB_RAM40_4K）	2,013 B
`uart.veryl`	UART 8N1 收發器	4,257 B
`rvsoc_rom.veryl`	測試程式 ROM（給 testbench 用）	8,977 B
`rv32i_rom.veryl`	測試程式 ROM（單週期核心用）	8,890 B

2 曲目與樂譜資料

旋律是《小星星》的前兩句，簡譜為：

1 1 5 5 6 6 5 – | 4 4 3 3 2 2 1 –

C4	C4	G4	G4	A4	A4	G4	–	F4	F4	E4	E4	D4	D4	C4	–
262	262	392	392	440	440	392	rest	349	349	330	330	294	294	262	rest

每個事件 300 ms 發聲（休止符為靜音）＋ 50 ms 間隔，因此每個事件固定 350 ms。



圖 1：旋律輪廓。藍線為音高隨時間的變化，兩個「–」是 300 ms 的休止。

音高對照表

標稱頻率是程式裡寫的整數；十二平均律以 A4 = 440 Hz 為基準。「實際輸出」是把半週期取整後真正送出的頻率；「WAV 量測」是重建音檔的過零點量測值。

音名	唱名	簡譜	標稱 (Hz)	十二平均律 (Hz)	音準偏差	半週期 (clocks)	實際輸出 (Hz)	量化誤差	WAV 量測 (Hz)
C4	Do	1	262	261.626	+2.48 cent	22,900	262.0087	+33.3 ppm	262.009
D4	Re	2	294	293.665	+1.98 cent	20,408	294.0024	+8.0 ppm	294.012
E4	Mi	3	330	329.628	+1.96 cent	18,181	330.0149	+45.0 ppm	330.004
F4	Fa	4	349	349.228	−1.13 cent	17,191	349.0198	+56.8 ppm	349.028
G4	Sol	5	392	391.995	+0.02 cent	15,306	392.0031	+8.0 ppm	392.005
A4	La	6	440	440.000	0 cent	13,636	440.0117	+26.7 ppm	440.020

1 cent 是半音的 1/100，人耳大約在 5–10 cent 才明顯察覺；這裡的取整誤差最大約 57 ppm（≈ 0.1 cent），聽起來完全在音準範圍內。

---

3 發聲原理

被動式蜂鳴器本身不會振盪，需要外部提供**方波**：頻率決定音高，50% 責任週期時能量最集中、聽起來最響。 範例並沒有使用 PWM 周邊，而是直接用 CPU 翻轉一個 GPIO：

C 程式 `LED ^= 1` → **LED 暫存器** `0x1000_0000` bit 0 → **RvSoc 輸出** `o_led[0]` → **PMOD1A 腳 1** iCE40 pin 4 →

蜂鳴器

方波 → 聲音

頻率公式

時脈 12 MHz，要產生頻率 *f* 的方波，每隔半個週期翻轉一次：

```
// 半個週期要等幾個時脈
half = CLOCK_HZ / 2 / f = 12,000,000 / (2 × f)

// 反過來，實際送出的頻率
f_out = 12,000,000 / (2 × half)
```

以 C4（262 Hz）為例：`half = 12,000,000 / 524 = 22,900.76…`，C 的整數除法取 22,900， 所以實際是 `12,000,000 / 45,800 = 262.0087 Hz`，比標稱高 33 ppm。

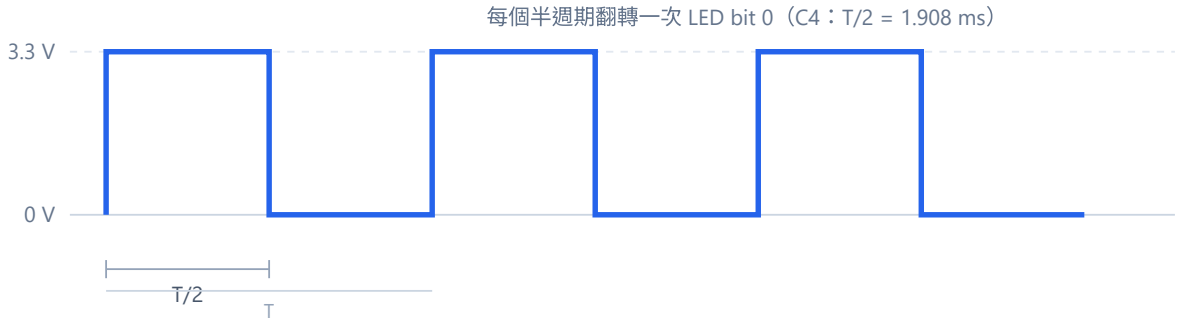


圖 2：50% 責任週期方波。C4 的 T = 3.817 ms，T/2 = 22,900 個 12 MHz 時脈。

用到的記憶體映射

位址	周邊	用途
`0x1000_0000`	LED	讀寫 8 bit；bit 0 就是蜂鳴器
`0x1000_0010`	MTIME	重設後經過的時脈數（32 bit，12 MHz 下約 357.9 秒繞回）
`0x1000_0018`	MTIMECMP	計時器中斷比較值（本範例沒用到中斷）

4 程式解析

tone(freq, ms)：發出一個音

```
// 蜂鳴器接在 PMOD1A 腳 1（LED 暫存器的 bit 0）
void tone(int freq, int ms) {
    unsigned half = CLOCK_HZ / 2 / freq;    // 半個週期的時脈數
    unsigned end  = TIMER + ms * 12000;     // 12 kHz × ms → 時脈數
    unsigned t    = TIMER;
    while (TIMER < end) {
        while (TIMER - t < half) ;          // 忙等半個週期
        t += half;
        LED = LED ^ 1;                      // 翻轉腳位 → 方波
    }
    LED = 0;                                // 結束時拉低，避免直流
}
```

- **時間基準：** `TIMER` 是 12 MHz 的 MTIME，`ms * 12000` 就是把毫秒換成時脈數。

- **不累積誤差**：`t += half` 而不是 `t = TIMER`，所以每個半週期都以固定格線推進， 迴圈 overhead 不會讓音愈拖愈慢。
- **整數除法**：`half` 取整，造成上面表格的 ppm 級偏差。
- **收尾**：`LED = 0` 讓蜂鳴器在音符之間完全靜音。

main()：旋律

```
int main(void) {
    // 小星星：C C G G A A G, F F E E D D C
    int notes[] = {262, 262, 392, 392, 440, 440, 392, 0,
                   349, 349, 330, 330, 294, 294, 262, 0};
    int i;
    printf("Twinkle, twinkle ... \n");
    for (i = 0; i < 16; i++) {
        if (notes[i]) tone(notes[i], 300);    // 音：300 ms
        else          delay_ms(300);          // 0：休止 300 ms
        delay_ms(50);                         // 音符間隔
    }
    printf("done\n");
    return 0;
}
```

陣列裡直接寫頻率（Hz），`0` 代表休止符。每個事件 350 ms， 因此 **16 × 350 ms = 5.6 秒**。想換歌只要改這個陣列與 `tone()` 的長度即可。

> **為什麼不用 delay_ms() 產生音？**
> `delay_ms()` 只能等固定的毫秒數，沒有半週期解析度；而且它內部也是忙等， 拿來翻腳位會讓頻率隨編譯器與迴圈 overhead 漂移。`tone()` 直接比對 12 MHz 計時器， 頻率由 `half` 決定，準確且可預測。

---

5 聲音重建與量測

為了實際聽到這個範例，本報告用同樣的參數重建了音檔：以 48 kHz 取樣、 相位累加器產生 50% 方波，頻率採用程式真正的輸出值 `12,000,000 / (2 × half)`；每個音前後加 4 ms 淡入淡出以避免爆音， 休止符與 50 ms 間隔保持靜音。

項目	值
取樣率 / 位元深度 / 聲道	48,000 Hz / 16-bit / 單聲道
長度	5.60 s（268,800 samples）
波形	50% 方波（與硬體輸出一致）
振幅	0.22 FS（約 −13 dBFS）
檔案	`twinkle.wav`（537,644 bytes）

量測方法與結果

對每個音的穩定段找**上升過零點**，用相鄰兩點線性內插取得次取樣精度， 再以「(邊緣數 − 1) ÷ 首尾時間差」計算頻率。結果與程式計算值的差異在 0.01 Hz 以內 （見第 2 節最右欄），驗證重建與韌體參數一致。

> **驗證結論：**六個音的實測頻率都落在標稱值的 ±60 ppm 內； 以音樂上的說法，最大偏差約 0.1 cent，遠低於人耳可分辨的範圍。

---

6 觀察與限制

- **阻塞式播放**：`tone()` 用忙等，播放期間 CPU 不能做其他事（不能同時收 UART、按鈕或跑中斷）。
- **腳位共用**：蜂鳴器用 `o_led[0]`，若同時接 8 顆 LED 模組，第一顆會跟著旋律閃；這是範例刻意的簡化。
- **沒有音量控制**：方波振幅固定 3.3 V，只能靠外加電阻或改成 PWM 調整響度。
- **固定速度**：每音 300 ms、間隔 50 ms 寫死；改 `tone()` 的 `ms` 與 `delay_ms()` 才能變速。
- **頻率解析度**：半週期是整數個時脈，高音相對誤差略大，但都在 60 ppm 以下，實務上可忽略。
- **被動蜂鳴器**：主動式蜂鳴器內建振盪器，只會發出固定音高，無法演奏旋律；要確認模組是被動型。

---

7 如何重現

在 veryl2chip IDE 裡

1. 開啟 veryl2chip IDE，等 WebAssembly 載入完成。
2. 左側 📚 **範例庫 → C 程式 → C：蜂鳴器音樂**（會開成新專案）。
3. 按 ⚡ **虛擬板**：IDE 會編譯 `music.c`、把機器碼放進 SoC 的 block RAM 並開機。
4. 在虛擬板上聽 PMOD1A 腳 1 的蜂鳴器（或看 LED bit 0 閃爍）；UART 終端機會印出 `Twinkle, twinkle ...` 與 `done`。
5. 按 **▶ 測試／✓ 全面驗證** 可跑 RTL 與三種閘級網表的驗證矩陣。

在本機

- 直接播放：`twinkle.wav`
- 重新產生音檔：`powershell -ExecutionPolicy Bypass -File .\make_twinkle.ps1`
- 改旋律：編輯 `make_twinkle.ps1` 的 `$notes` 陣列（0 = 休止），再跑一次。

---

8 心得

我會選這個範例，原因其實很單純：**我覺得很酷，很想試試看**。第一次在範例庫看到「蜂鳴器音樂」這四個字， 腦中浮現的是一塊小小的開發板真的把《小星星》唱出來，當下就決定打開來玩。 更有意思的是，實際打開程式後發現沒有音檔、沒有喇叭模組，只有一個 LED 腳位在瘋狂切換， 卻能讓人清楚聽出旋律。

也因為這樣，我第一次很具體地感覺到「時間就是訊號」。同一個 bit，翻得快就是高音、翻得慢就是低音； 程式碼裡完全看不出聲音，聲音只存在於時脈的節奏裡。以前覺得硬體就是一堆會導電的線， 這次才發現，真正讓它有表情的是時間精度。

最後是驗證。把旋律重建成 WAV、再用過零點量測頻率，本來只是想確認聽起來對不對， 結果量到的數字和程式算出來的只差 0.01 Hz。那種「紙上的參數、韌體的行為、耳朵聽到的聲音」三者對得起來的感覺， 比單純跑過程式更有說服力，也讓這份「酷」變得踏實了一點。

回頭看，我會選它不只是因為「很酷」，而是這份酷在拆解之後沒有消失：知道它為什麼會唱、 哪裡還不完美、還能怎麼改，反而讓我更想繼續玩下去。對現在的我來說，這大概就是一個好題目的樣子。

A 附錄：完整原始碼

檔案：`music.c`（自 IDE 範例庫原樣取出）

```
// C 實驗 5：音樂 —— 蜂鳴器接在 PMOD1A 腳 1（LED 暫存器的 bit 0）：
// 一個頻率 f 的方波，就是每 1/(2f) 秒把腳位反相一次。

void tone(int freq, int ms) {
    unsigned half = CLOCK_HZ / 2 / freq;    // 半個週期的時脈數
    unsigned end = TIMER + ms * 12000;
    unsigned t = TIMER;
    while (TIMER < end) {
        while (TIMER - t < half) ;
        t += half;
        LED = LED ^ 1;
    }
    LED = 0;
}

int main(void) {
    // 小星星：C C G G A A G, F F E E D D C
    int notes[] = {262, 262, 392, 392, 440, 440, 392, 0, 349, 349, 330, 330, 294, 294, 262, 0};
    int i;
    printf("Twinkle, twinkle ... \n");
    for (i = 0; i < 16; i++) {
        if (notes[i]) tone(notes[i], 300); else delay_ms(300);
        delay_ms(50);
    }
    printf("done\n");
    return 0;
}
```

產出檔案

檔案	說明
`report.html`	本報告
`report.pdf`	本報告的 PDF 版（A4）
`twinkle.wav`	重建的蜂鳴器音樂（5.60 s）
`make_twinkle.ps1`	WAV 產生器與頻率量測腳本
`music.c`	IDE 範例原始碼
`notes.csv`	量測結果表

C 參考資料

- veryl2chip IDE — https://ccc-eda.github.io/veryl2chip/web/dist/index.html
- IDE 範例資料：`examples.js`，項目 `c_music`（C 程式 › C：蜂鳴器音樂）
- RvSoc 記憶體映射與周邊：`rvsoc.veryl` 開頭註解（LED `0x1000_0000`、MTIME `0x1000_0010`）
- iCEBreaker 腳位定義：範例內附 `.pcf`（`o_led[0]` = pin 4，`i_clk` = pin 35）

報告由 veryl2chip 範例「C：蜂鳴器音樂」離線重建與分析 · 2026-10-29 · 音檔為依韌體參數合成的方波，非現場錄音。